



Actividad práctica en parejas

Objetivo: tomar el conversor de la clase pasada, que atiende dos botones con un solo receptor y un `if`, y pasar por las tres formas de escribir un receptor, clase con nombre, clase anónima y `lambda`, hasta dejar un receptor por evento. Después atender el teclado y el mouse con adaptadores, y comprobar que una moneda nueva entra sin modificar nada de lo que ya estaba.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es `10-lambdas/actividad`.

Es el conversor de pesos a dólares y euros tal como quedó al final de la clase pasada, repartido en `modelo`, `vista` y `controlador`. Casi todo el trabajo de hoy es en `ControladorConversor.java`. `Programa.java` no se modifica salvo comentar y descomentar los bloques y el `import` que indica cada parte.

Parte 1: ver el problema

Ejecutar `Programa` tal como viene y probar los dos botones con 1450. Leer `ControladorConversor.java` completo. Responder por escrito.

1. El contador trae la cotización del real. Para agregar el botón “A reales”, ¿qué hay que tocar en `actionPerformed()`? ¿Qué principio de SOLID se rompe al modificar un método que ya funcionaba?
2. `actionPerformed()` atiende dos botones. ¿Cuántas cosas distintas decide ese método hoy? ¿Y con cinco monedas?
3. El cajero pide que la tecla `Enter`, en el campo, convierta a dólares. `KeyListener` es la interfaz para el teclado y tiene tres métodos: `keyTyped()`, `keyPressed()` y `keyReleased()`. ¿Se puede atender el `Enter` agregando un caso al `if` de `actionPerformed()`? ¿Por qué no?
4. Si se escribe una clase `ReceptorEuros` implements `ActionListener` en su propio archivo, ¿qué necesita tener adentro para poder hacer lo que hoy hace la rama de euros? Anotar la lista de atributos.



Parte 2: un receptor por evento

Un TODO por vez, en orden. El proyecto compila al terminar cada uno, y Programa no se toca hasta la parte 3. Después de cada TODO, ejecutar y comprobar que los dos botones siguen funcionando.

- TODO 1: escribir `ReceptorEuros`, una clase con nombre en `controlador/`, y registrarla como receptor del botón de euros. Sacar el `if` del controlador, que pasa a atender solo dólares.
- TODO 2: reemplazar `ReceptorEuros` por una clase anónima escrita en el constructor del controlador, que llame a un método privado `convertirAEuros()`. Borrar `ReceptorEuros.java`.
- TODO 3: reemplazar los dos receptores por una lambda por botón, cada una llamando a su método privado. Sacar `implements ActionListener` y `actionPerformed()` del controlador.
- TODO 4: hacer que Enter en el campo convierta a dólares. Probar primero con una lambda y leer lo que dice el compilador.
- TODO 5: hacer que un doble click en el campo vacíe el campo y el resultado.

Con los cinco TODO hechos, la ventana se ve igual que al principio, y responde al teclado y al mouse.

Parte 3: la moneda nueva

- TODO 6: agregar el botón “A reales”, con una cotización de 265 pesos por real, sin modificar ninguna línea que ya existía en el controlador. Pasar Programa al bloque de la parte 3a y probar con 1450 en los tres botones.
- TODO 7: pasar Programa al bloque de la parte 3b, con su `import`, que arranca la ventana desde el hilo de Swing con `invokeLater()`. Se ve igual.

Antes de seguir, anotar qué se cambió en cada clase por el TODO 6 y cuántas líneas que ya existían hubo que modificar en el controlador. Comparar con la pregunta 1 de la parte 1.

Para discutir en la pareja y anotar:

- `ReceptorEuros.java` tenía unas veinte líneas. ¿Cuántas hacían algo que el controlador no hacía ya? ¿Cuántas quedaron en la lambda del TODO 3?
- Con cinco monedas, el constructor tendría cinco lambdas y el controlador cinco métodos privados con la misma forma. ¿Es el mismo problema que el `if` de la parte 1? ¿Qué se modifica en cada caso al agregar la sexta?
- El `KeyListener` del TODO 4 no pudo ser una lambda, y las de los botones sí. ¿Qué tiene `ActionListener` que `KeyListener` no tiene?
- En el TODO 2, ¿por qué `this.convertirAEuros()` no compila adentro de la clase anónima, y sí compila adentro de la lambda del TODO 3?
- Con cinco botones y una ventana que el usuario quiere agrandar, ¿qué pasa con las coordenadas fijas? Cómo ubicar componentes sin coordenadas es el tema de la clase que viene.